TEAM NUMBER: 131

AUTOMATIC CATEGORY SUGGESTIONS FOR WIKIMEDIA COMMONS IMAGES

STEP 1. PROBLEM STATEMENT

A small program. You give it a photo. It tells you category names for that photo. photo of a cat -> our program -> Cats, Pets, Animals.Wikipedia's image website (Wikimedia Commons) has millions of photos. People add categories by hand, so many photos have no category and are hard to find. Our program suggests categories. A person then says yes or no. The catch is one image can fall under various categories. Each category is ranked according to its relevance. Moreover, the absence of a parent category from an image's existing labels doesnt mean that the parent is incorrect. We treat missing labels as unknown rather than saying they are incorrect.

We basically build a system that can search for relevant categories instead of checking every possible category individually.

STEP 2. How do we do it?

We are using already trained AI models like CLIP ,it already knows photos and words.We give CLIP a list of words, like cat, dog, temple, beach. CLIP tells us which words fit the photo best. We collected a large dataset from wikimedia commons to use it for our new model.

Step 3 What we did

1. Made a GitHub repo. This is an online folder for our code. One person made it and added the others. We use the github repo to make the code available for all.

2. we Opened Google Colab and went to colab.research.google.com and clicked New notebook. It runs Python for free. Nothing to install.

3. We tested CLIP on one photo with the code below. We Uploaded a photo named test.jpg and ran it.

CODE:

```
from transformers import CLIPModel, CLIPProcessor from PIL import Image import torch

model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32") processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

categories = ["cat", "dog", "temple", "beach", "football match", "Diwali festival", "mountain", "train", "food", "portrait"]

image = Image.open("test.jpg") inputs = processor(text=[f"a photo of {c}" for c in categories], images=image, return_tensors="pt", padding=True)

with torch.no_grad(): out = model(**inputs)

probs = out.logits_per_image.softmax(dim=1)[0] ranked = sorted(zip(categories, probs.tolist()), key=lambda x: -x[1]) for name, p in ranked[:5]: print(name, round(p, 3))
```

STEP 4.

Everyone in the team implemented step 3. So everyone understood the basics.

STEP 5:

We downloaded random images from wikimedia commons page and uploaded them on google collab and verified the result it gave.

STEP 6.

We looked at the top 5 suggestions and counted how many are correct. This score is called Precision@5.

STEP 7. We added the "find similar photos" way and the "small helper" way. Then mix them.

STEP 8.We will make a demo page. A simple page where you upload a photo and see the suggestions. Use a tool called Gradio.